DOM Continuation

Lec-11

Creating New HTML elements using DOM

- To add a **new element to the HTML DOM**, you must **create the element (element node) first**, and **then append it to an existing element**.

1. document.createElement()

- This method generates a new HTML element specified by its tag name. When you call this, the element **only exists in the computer's memory—it is not yet visible on the screen.**

Eg :

```
const heading1 = document.createElement('h1')
console.log(heading1);

// Output: <h1></h1>
```

2. document.createTextNode()

- This method creates a plain text node. It is a safe way to handle text because it automatically escapes any HTML tags, preventing Cross-Site Scripting (XSS) vulnerabilities. Like `createElement()`, t**his node only exists in memory until attached.**

Eg :

```
const h1TextNode = document.createTextNode('Hello world')
```

3. appendChild()

- This method takes a node and adds it to the **end of the children list of a specified parent element. You use this to link your elements together in memory, and finally to hook them to the visible webpage layout** (like appending to `document.body`).

```
const h1TextNode = document.createTextNode('Hello world')
heading1.appendChild(h1TextNode)
document.body.appendChild(heading1)

// Output: <h1>Hello world</h1>
```

Hello world

Creating New HTML elements using DOM

4. insertBefore()

- The **insertBefore()** method in the JavaScript DOM is **used to insert a new node before an existing child node inside a specified parent element.**
- While **appendChild()** always adds an element to the very end, **insertBefore()** gives you precise control over the exact placement.

HTML

```
<body>
    <ul>
      <li>Egg</li>
      <li>Milk</li>
      <li>Bread</li>
    </ul>
</body>
```

Adding a new li as Coffe before milk using JS DOM

```
// Creating new li
let listItem = document.createElement('li')

// Creating Text node for li
let coffee = document.createTextNode('Coffee')

// appending text node to li
listItem.appendChild(coffee)
console.log(listItem);

// selecting the element before which we'll be adding our new element
let list = document.querySelector('li:nth-child(2)')
console.log(list);

// selecting the parent element in which siblings exists
let ul = document.querySelector('ul')

// inserting our newly created element
ul.insertBefore(listItem,list)
```

Editing HTML Elements using DOM

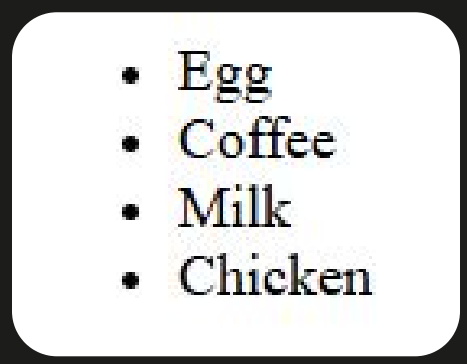
5. replaceWith()

- The `replaceWith()` method in the JavaScript DOM is **used to replace an existing element with one or more new elements or text nodes**.
- Suppose you have the following list and you wish to replace **Bread** with **Chicken.**


```
// selecting element which you want to replace
let itemTobeReplaced = document.querySelector('li:last-child')
console.log(itemTobeReplaced);

// creating and element which will be replacing the older element
let replacingElement = document.createElement('li')
replacingElement.textContent = 'Chicken'

// replacing the old element with new one using replaceWith()
itemTobeReplaced.replaceWith(replacingElement)
```



Remove HTML Element using DOM

6. remove() : The `remove()` method in the JavaScript DOM is **used to completely delete an element from the web page**

suppose we want to remove **Milk** from our list,

We can remove it in following way:

```
// selecting item to be removed
let itemToBeRemoved = document.querySelector('li:nth-child(3)')
console.log(itemToBeRemoved);

// removing the item using .remove()
temToBeRemoved.remove()
```